

Context Tuning for In-Context Optimization

Jack Lu¹ Ryan Teehan¹ Zhenbang Yang¹ Mengye Ren¹

Abstract

We introduce *Context Tuning*, a simple and effective method to significantly enhance few-shot adaptation of language models (LLMs) without fine-tuning model parameters. While prompt-based adaptation techniques have demonstrated the effectiveness of lightweight adaptation methods for LLMs, they typically initialize a trainable prompt or prefix with irrelevant tokens for the task at hand. In contrast, *Context Tuning* initializes the trainable prompt or prefix with task-specific demonstration examples, leveraging the model’s inherent In-Context Learning (ICL) ability to extract relevant information for improved few-shot learning performance. Extensive evaluations on benchmarks such as CrossFit, UnifiedQA, MMLU, BIG-Bench Hard, and ARC demonstrate that *Context Tuning* outperforms traditional prompt-based adaptation methods and achieves competitive accuracy with Test-Time Training at significantly higher training efficiency.

1 Introduction

Large language models (LLMs) have demonstrated impressive capabilities across a wide range of natural language processing (NLP) tasks by leveraging knowledge acquired during large-scale pretraining (Brown et al., 2020; Grattafiori et al., 2024; Jiang et al., 2023). These models can adapt to new tasks using only a few input and output examples provided in context, a process known as In-Context Learning (ICL) (Brown et al., 2020). However, ICL often struggles with complex reasoning or domain shifts, as it relies solely on a forward pass to interpret the examples. While methods like Test-Time Training (TTT) (Akyürek et al., 2025) have shown that effective adaptation is

¹Agentic Learning AI Lab, New York University, New York, NY, USA. Correspondence to: Jack Lu <yl11330@nyu.edu>, Mengye Ren <mengye@nyu.edu>. Code is available at <https://github.com/agentic-learning-ai-lab/context-tuning>.

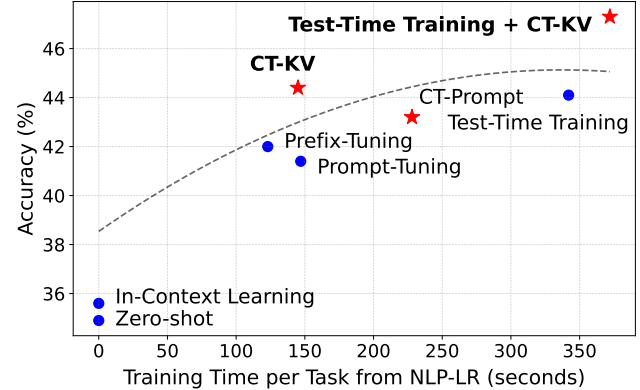


Figure 1. Comparison of training-free, prompt-based adaptation, and *In-Context Optimization* methods on solving 26 NLP-LR tasks from Table 1. Circles are baselines; stars are our methods; **bolded** methods attain the best performance-efficiency tradeoff.

possible with limited data, they can still be computationally expensive. This highlights the need for more efficient and effective approaches to task adaptation in LLMs.

Contrary to ICL’s reliance on a forward pass, prompt-based adaptation methods like Prompt Tuning (Lester et al., 2021) and Prefix Tuning (Li & Liang, 2021) prepend a set of trainable vectors to each example input and optimize them via gradient descent. At a conceptual level, ICL harnesses the model’s ability to extract task-relevant information from the context of few-shot examples, while prompt-based adaptation methods optimize randomly initialized vectors to guide the model’s behavior in solving each example. Given these complementary approaches, it is natural to ask whether we can bridge them by directly optimizing the context of few-shot examples to steer the model more effectively.

In this work, we introduce *Context Tuning*, a simple and effective method for few-shot learning that initializes trainable vectors from the few-shot examples of a novel task, then optimizes them to solve each example. We develop two variants: *CT-Prompt*, which applies Prompt Tuning to optimize a soft prompt initialized from few-shot examples, and *CT-KV*, which applies Prefix Tuning to optimize the key-value (KV) cache derived from those same examples. While *CT-Prompt* achieves strong performance, it suffers from a quadratic training-time cost in the number of examples. Similarly, the recently proposed Test-Time

Training (TTT) (Akyürek et al., 2025) method, which fine-tunes model parameters with LoRA (Hu et al., 2022) on permutations of few-shot examples, also incurs quadratic cost. In contrast, *CT-KV* achieves linear training time complexity while also outperforming *CT-Prompt* and achieving competitive performance with TTT, due to the efficiency and per-layer conditioning of the KV cache. In addition, because *Context Tuning* tunes the context and TTT tunes the model, the two methods are complementary: applying *CT-KV* to refine the model context after TTT’s weight updates leads to additional performance gains. A high-level comparison in Figure 1 illustrates *CT-KV*’s high efficiency and accuracy, whether used alone or in combination with TTT.

We situate two approaches for few-shot learning with in-context examples within a broader framework we term *In-Context Optimization (ICO)*: TTT optimizes the model itself, while *Context Tuning* optimizes the model’s context. Under this framework, adaptation leverages the LLM’s ICL ability and either updates its parameters or its context representation. We evaluate ICL, prompt-based adaptation methods, and *ICO* techniques across a wide range of natural language and symbolic reasoning benchmarks, including CrossFit (Ye et al., 2021), UnifiedQA (Khashabi et al., 2020), BIG-Bench Hard (BBH) (Srivastava et al., 2023; Suzgun et al., 2023), MMLU (Hendrycks et al., 2021), and the Abstraction and Reasoning Corpus (ARC) (Chollet, 2019b). *CT-KV* significantly outperforms both ICL and prompt-based adaptation methods, while remaining competitive with the more computationally intensive TTT approach. Furthermore, we show that *CT-KV* can serve as a post-hoc refinement step following TTT, leading to improved few-shot adaptation performance compared to either method used in isolation.

2 Related Work

Prompt-Based Adaptation. Prompt-Based Adaptation steers pretrained language models to solve downstream tasks by learning task-specific inputs while keeping the model weights frozen. AutoPrompt (Shin et al., 2020) was an early method that constructed discrete prompts via gradient-based search. Prefix Tuning (Li & Liang, 2021) introduced trainable continuous vectors that serve as a prefix to the model’s key-value cache at each layer, achieving strong performance on generation tasks with only a small number of trainable parameters. P-Tuning (Liu et al., 2024d) appended soft prompts to the input and used an LSTM-based prompt encoder to model dependencies between prompt tokens. Prompt Tuning (Lester et al., 2021) simplified the approach by learning soft prompts solely at the input layer and demonstrated that performance improves with model scale. P-Tuning v2 (Liu et al., 2021) provided an optimized implementation of Prefix Tuning and extended it to natural language understanding tasks. While these works

typically initialize their learnable prompts using high-level task instructions, random tokens, or unrelated words, *Context Tuning* leverages the pretrained LLM’s ability to extract meaningful task-specific information directly from in-context demonstration pairs. Finally, Singhal et al. (2023) proposed Instruction Prompt Tuning, in which expert-curated few-shot demonstrations are prepended to a learned soft prompt. In contrast, *Context Tuning* draws demonstration pairs directly from the dataset and uses them to initialize the prompt rather than prepending them as input.

In-Context Learning. Introduced by Radford et al. (2019), ICL has become a defining feature of large language models (LLMs), enabling them to perform novel tasks by conditioning on a few input-output demonstrations without any parameter updates. This behavior has been leveraged through various prompting strategies, such as Chain-of-Thought prompting to elicit reasoning (Wei et al., 2022) and self-consistency decoding to reduce variance (Wang et al., 2023). Prior work has also explored selecting informative demonstrations (Liu et al., 2022; Li & Qiu, 2023), as well as meta-training over large sets of tasks to improve ICL generalization and inference-time efficiency (Min et al., 2022a; Chen et al., 2022; Pan et al., 2023; Huang et al., 2023; Qin et al., 2023; Muhtar et al., 2024). From a theoretical perspective, Dai et al. (2023) and Deutch et al. (2024) interpret ICL as performing implicit gradient descent; Zhao (2023) conceptualizes it as contextual retrieval within an associative memory framework; and Garg et al. (2022) demonstrates that transformers trained from scratch can learn complex function classes in-context. While these findings highlight ICL’s potential, recent studies by Min et al. (2022b) and Jang et al. (2024) show that LLMs often only rely on superficial patterns in the demonstrations rather than learning the underlying task. In Section 5.9, we further investigate these limitations by analyzing the intermediate key-value (KV) cache extracted from demonstration pairs, showing that it fails to encode sufficient task information. We address this shortcoming through gradient optimization.

Inference-Time Optimization. Our framework, *In-Context Optimization*, contributes to a broader class of methods that adapt models or their internal representations at inference time. Originally applied to image classification (Sun et al., 2020; Gandelsman et al., 2022), test-time training has since shown strong results in language modeling (Hardt & Sun, 2024), video generation (Dalal et al., 2025), controllable language generation (Liu et al., 2024b), and abstract reasoning (Bonnet & Macfarlane, 2024). In diffusion models (Ho et al., 2020; Rombach et al., 2022), techniques such as classifier guidance and classifier-free guidance (Dhariwal & Nichol, 2021; Ho & Salimans, 2022) steer generation by optimizing intermediate outputs during sampling. These methods have enabled controllable text-to-image synthesis (Nichol et al., 2022), adjustable

aesthetic attributes (Wallace et al., 2023), and improved sample diversity (Lu et al., 2024). More recently, Akyürek et al. (2025) proposed test-time training of LoRA (Hu et al., 2022) parameters for ICL using a leave-one-out strategy, achieving state-of-the-art performance on the Abstraction and Reasoning Corpus (ARC) (Chollet, 2019b;a). In contrast, *Context Tuning* tunes a soft prompt or continuous prefix rather than updating model weights, and we evaluate it on a broader range of ICL tasks.

3 Background

We introduce the mathematical formulation of ICL, Prefix Tuning, and Prompt Tuning. To set up the problem of single-task few-shot adaptation, we consider a language model p_ϕ with parameters ϕ , d hidden dimensions, L layers, a demonstration set $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^k$, and the goal of solving a new query x_q from the same task. We denote the concatenated context of all demonstration pairs as $\mathcal{C} = [x_1; y_1; \dots; x_k; y_k]$.

In-Context Learning. ICL concatenates all k demonstration pairs followed by the query x_q . The model then predicts $\hat{y}_q$ conditioned on this context: $\hat{y}_q = \arg \max_y p_\phi(y \mid [\mathcal{C}; x_q])$. In ICL, there is no gradient-based optimization; instead, the model adapts by attending to the tokens of the demonstration pairs provided in context.

Prompt Tuning. In Prompt Tuning, the model parameters ϕ remain fixed. Instead, m trainable soft prompt tokens P are prepended to each input and optimized via gradient descent:

$$P^* = \arg \min_P \sum_{i=1}^k -\log p_\phi(y_i \mid [P; x_i]). \quad (1)$$

After optimizing on the demonstration pairs, the optimized soft prompt P^* can be used for inference: $\hat{y}_q = \arg \max_y p_\phi(y \mid [P^*; x_q])$.

Prefix Tuning. Prefix Tuning also keeps ϕ fixed but learns layer-wise prefixes of m trainable vectors for the keys and values in each transformer layer: $\Theta = \{K_j, V_j\}_{j=1}^L$. Each layer’s attention uses these prefixes by prepending K_j to its keys and V_j to its values. The prefixes are optimized to minimize

$$\Theta^* = \arg \min_\Theta \sum_{i=1}^k -\log p_\phi(y_i \mid [\Theta; x_i]). \quad (2)$$

After obtaining Θ^* , inference on the query x_q proceeds analogously to Prompt Tuning.

4 Context Tuning for In-Context Optimization

In this section, we introduce the mathematical formulation of *In-Context Optimization (ICO)*, a few-shot adaptation scheme that uses demonstrations in the context and performs gradient-based optimization on either the model parameters or a context representation. We then show that Test-Time Training (TTT) (Akyürek et al., 2025) is an instance of *ICO*. Finally, we present Context Tuning, formalizing its *CT-Prompt* and *CT-KV* variants along with the two additional design choices that drive their strong performance.

4.1 In-Context Optimization

To combine the strengths of supervised fine-tuning and LLMs’ inherent ability to learn from context, *ICO* unifies two prevalent techniques for few-shot learning: ICL and gradient-based optimization. Formally, the objective of *ICO* is minimizing the loss

$$\sum_{i=1}^k -\log p_\phi(y_i \mid [\theta_{\text{context}}^{(i)}; x_i]), \quad (3)$$

where $\theta_{\text{context}}^{(i)}$ is a context representation derived from the set of demonstration pairs $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^k$. One may notice that this objective resembles Equations 1 and 2 because traditional prompt-based adaptation methods also prepend additional contexts to inputs during optimization. Still, these contexts are randomly initialized instead of utilizing the demonstration pairs $\mathcal{D}$. Therefore, Prompt Tuning and Prefix Tuning are not instances of *ICO* by definition. Furthermore, since ICL does not perform gradient-based optimization at all, it also does not fall under *ICO*.

4.2 Test-Time Training as ICO

TTT (Akyürek et al., 2025) can be viewed as an instance of *ICO*. Specifically, TTT minimizes Equation 3 by first initializing the model weights ϕ from a pretrained model, then updating them with LoRA layers for parameter efficiency. At each optimization iteration, TTT dynamically sets $\theta_{\text{context}}^{(i)} = \mathcal{C}^{-i}$, where $\mathcal{C}^{-i}$ represents the concatenated tokens of a random permutation of demonstration pairs except for the i -th pair. Therefore, the optimization equation becomes: $\phi^* = \arg \min_\phi \sum_{i=1}^k -\log p_\phi(y_i \mid [\mathcal{C}^{-i}; x_i])$. To perform inference on the query input x_q , TTT uses the optimized model weights and the concatenation of all demonstration pairs as context: $\hat{y}_q = \arg \max_y p_{\phi^*}(y \mid [\mathcal{C}; x_q])$.

4.3 Context Tuning

We design our *Context Tuning* approach to be an instantiation of the *ICO* framework. In contrast to TTT, *Context Tuning* freezes model parameters ϕ and instead directly

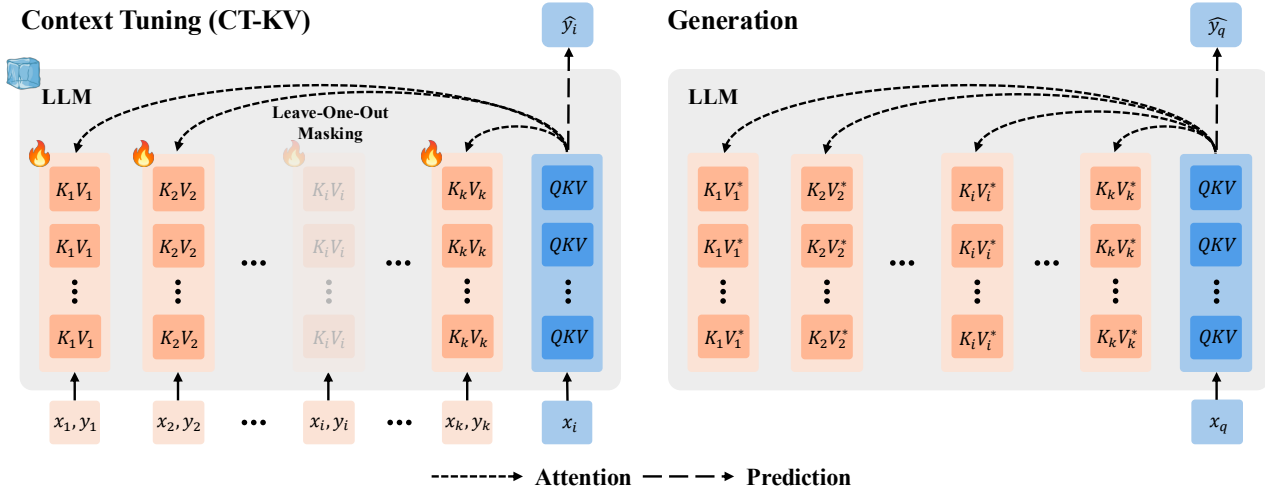


Figure 2. *CT-KV*, the variant of *Context Tuning* that optimizes the key-value prefixes derived from in-context demonstration pairs. *CT-KV* (left) first initializes a prefix $\{K_i, V_i\}_{i=1}^k$ from demonstration pairs $\{(x_i, y_i)\}_{i=1}^k$, then trains it to solve each pair. To prevent the model from simply retrieving the demonstration pair from the prefix, *Leave-One-Out Masking* prevents the model from attending to K_i, V_i when solving pair i . At generation time (right), the model conditions on all optimized prefixes $\{K_i^*, V_i^*\}_{i=1}^k$ to solve query x_q .

optimizes the lightweight context representation θ_{context} of the demonstration pairs.

- *CT-Prompt* initializes $\theta_{\text{context}} = P_{\text{CT}}$ as the model’s prompt embeddings on $\mathcal{C}$, the concatenation of demonstration pairs.
- *CT-KV* initializes $\theta_{\text{context}} = \Theta_{\text{CT}}$ as a key-value prefix $\Theta_{\text{CT}} = \{K_j, V_j\}_{j=1}^L$ obtained from the model’s layer-wise activations on $\mathcal{C}$.

Furthermore, we introduce two design choices for both *CT-Prompt* and *CT-KV*. We study the performance impact of each in Section 5.7, demonstrating that both are crucial for achieving strong empirical gains.

Leave-One-Out Masking. To prevent the model from simply retrieving the answer y_i of the i -th demonstration pair embedded in θ_{context} when predicting the output for x_i , we construct

$$\theta_{\text{context}}^{(i)} = \begin{cases} P_{\text{CT}}^{-i} & \text{for } CT\text{-Prompt}, \\ \Theta_{\text{CT}}^{-i} & \text{for } CT\text{-KV}, \end{cases}$$

and use it instead of θ_{context} in optimization. When conditioning on P_{CT}^{-i} or Θ_{CT}^{-i} , the trainable soft prompt tokens in *CT-Prompt* or prefix tokens in *CT-KV* corresponding to the in-context demonstration pair (x_i, y_i) are masked out from the attention view of the model by simply setting the corresponding positions of the attention mask to 0. In contrast to TTT’s leave-one-out technique, which omits one demonstration pair in the context to update the

model weights, our *Leave-One-Out Masking* operates on the derived context vectors with the model parameters frozen, ensuring that the optimization refines the context representation itself rather than relying on weight updates.

Token Dropout. Since *Context Tuning* generally introduces a larger number of prompt or prefix tokens than traditional prompt-based adaptation techniques, we regularize training by randomly dropping tokens in $\theta_{\text{context}}^{(i)}$ with a fixed probability, denoted as *TokenDrop*. During optimization, the loss is computed in expectation over these stochastic dropout masks, encouraging the learned context to avoid overfitting to any single token.

Altogether, we arrive at the following optimization equation for *CT-Prompt*:

$$P_{\text{CT}}^* = \arg \min_{P_{\text{CT}}} \sum_{i=1}^k -\log p_{\phi} \left(y_i \mid [\text{TokenDrop} \left(P_{\text{CT}}^{-i} \right); x_i] \right),$$

and the following for *CT-KV*:

$$\Theta_{\text{CT}}^* = \arg \min_{\Theta_{\text{CT}}} \sum_{i=1}^k -\log p_{\phi} \left(y_i \mid [\text{TokenDrop} \left(\Theta_{\text{CT}}^{-i} \right); x_i] \right).$$

To perform inference on the query x_q , *CT-Prompt* and *CT-KV* use their respective optimized contexts. *CT-Prompt*’s inference becomes $\hat{y}_q = \arg \max_y p_{\phi} (y \mid [P_{\text{CT}}^*; x_q])$ and *CT-KV*’s inference becomes $\hat{y}_q = \arg \max_y p_{\phi} (y \mid [\Theta_{\text{CT}}^*; x_q])$.

In practice, *CT-Prompt* requires recomputing layer-wise keys and values corresponding to P_{CT} , while *CT-KV* does

not. In Appendix A, we formally prove that for each optimization step, *CT-KV* has lower time complexity than both TTT and *CT-Prompt* with respect to the number of demonstration pairs. Finally, we introduce TTT+*CT-KV*, which first performs TTT to update model weights ϕ , then applies *CT-KV* to refine the model’s demonstration context for improved performance.

5 Experiments

5.1 Datasets

We evaluate on a diverse set of challenging datasets for pretrained LLMs. We show a representative task example for each dataset in Figure 3.

- **NLP-LR** is the low-resource dataset split introduced by Min et al. (2022a), encompassing over 26 NLP tasks from CrossFit (Ye et al., 2021) and UnifiedQA (Khashabi et al., 2020), such as sentiment analysis and paraphrasing. Following Min et al. (2022a), we sample $k = 16$ demonstration pairs per task and evaluate task instances as multiple-choice problems.
- **Massive Multitask Language Understanding (MMLU)** is a diverse benchmark consisting of 57 subject-specific tasks, including mathematics, history, law, and various other domains (Hendrycks et al., 2021). We sample $k = 16$ demonstration pairs per task and evaluate task instances as multiple-choice problems.
- **BIG-Bench Hard (BBH)** is a curated subset of BIG-Bench, consisting of 23 tasks that challenge pretrained LLMs with questions involving algorithmic puzzles, symbolic manipulation, and other complex reasoning domains (Srivastava et al., 2023; Suzgun et al., 2023). Following Akyürek et al. (2025), we sample $k = 10$ demonstration pairs per task and prepend trainable instructions to all of our methods. We evaluate these tasks as question-answering problems.
- **Abstraction and Reasoning Corpus (ARC)** is a challenging symbolic reasoning benchmark with 400 evaluation tasks, each defined by a few grid transformation pairs and one or more query input grids (Chollet, 2019b). Since the average number of available demonstration pairs is fewer than 4, we use all of them in context. Tasks are evaluated as question-answering problems.

Each dataset is formatted either as a multiple-choice task or a question-answering task. For multiple-choice problems, where the LLM must select an output from a predefined set of answers, we follow Min et al. (2022a) and choose the option with the lowest loss. For question-answering tasks, the LLM has to generate an answer that matches the ground-truth output.

5.2 Models

Following Min et al. (2022a) and Akyürek et al. (2025), we use GPT-2 and Llama3-8B for NLP-LR and BBH, respectively. To demonstrate that *Context Tuning* performs well across different model sizes, we select Llama3.2-3B for MMLU. Due to computational constraints, we use Llama3.2-1B for ARC, which requires handling long input sequences. Since the pretrained Llama3.2-1B model cannot solve any of the 400 ARC evaluation tasks, we follow Akyürek et al. (2025) and Franzen et al. (2025) by fine-tuning it on the ARC training split, which contains 400 tasks that do not overlap with the evaluation split. While the works of Franzen et al. (2025) and Akyürek et al. (2025) focus on achieving high scores in the ARC competition (Chollet et al., 2024), our goal is to develop a method applicable across general few-shot problems. Therefore, we do not perform augmentation or voting for ARC. All pretrained model checkpoints were obtained from HuggingFace.

5.3 Experiment Setup

On top of zero-shot inference and ICL, we compare a variety of few-shot learning techniques to conduct a broad investigation of prompt-based adaptation strategies and methods under the *In-Context Optimization* framework: Prompt Tuning, Prefix Tuning, TTT, *CT-Prompt*, *CT-KV*, and TTT+*CT-KV*. We use greedy decoding for all question-answering tasks. All experiments in Table 1 are run over 5 different sets of randomly selected demonstration pairs, except for ARC, which has a fixed set of demonstration pairs for each task.

For Prompt Tuning and Prefix Tuning, we either set the number of trainable tokens m to 32, or match it to the number of tokens that are in the demonstration pairs used by *Context Tuning*. Trainable soft prompts and prefixes are initialized using sampled token embeddings from the model, which we find yields the best baseline performance.

For ARC, we fine-tune our Llama3.2-1B checkpoint following the setup of Franzen et al. (2025), using 2 A100 GPUs for 24 epochs with a learning rate of $2e-4$, cosine learning rate scheduler, 1 warmup epoch, and a global batch size of 32 (after gradient accumulation). All other experiments are conducted on a single A100 GPU, except NLP-LR, which is run on an RTX8000. For *CT-Prompt* and *CT-KV*, we apply Leave-One-Out Masking from Section 4 across all datasets, except ARC, where performance improves without it. We elaborate on this decision in Section 5.7.

For completeness, we also compare alternative setups for both Prompt Tuning and Prefix Tuning. In Appendix B, we report results using uniformly initialized trainable parameters for both methods. We also include results for Prefix Tuning with an MLP parameterization, along with details of our hyperparameter search to support

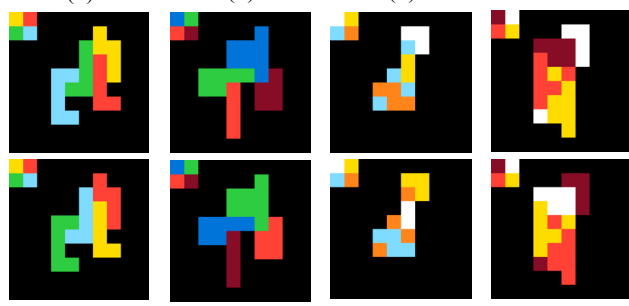
BBH Instruction: A logical deduction task which requires deducing the order of a sequence of objects. Answer with only the corresponding letter (e.g. (A)). Query: The following paragraphs each describe a set of three objects arranged in a fixed order. The statements are logically consistent within each paragraph. In a golf tournament, there were three golfers: Ada, Mel, and Mya. Mya finished below Ada. Mel finished above Ada. Options: (A) Ada finished last (B) Mel finished last (C) Mya finished last Answer: (C)	
NLP-LR Query: What would you measure in a graduated cylinder? Options: <i>nitrogen, Perfume, Oxygen, helium</i> Answer: <i>Perfume</i>	
MMLU Query: Find the degree for the given field extension $Q(\sqrt{2} + \sqrt{3})$ over Q. Options: 0, 4, 2, 6 Answer: 4	ARC (1) (2) (3) Test Query Answer 

Figure 3. One test pair from BBH, NLP-LR, and MMLU each, and 3 demonstration pairs followed by a test pair from ARC. BBH contains instructions that we prepend to model inputs. NLP-LR and MMLU contain multiple-choice options for the model to select. To avoid clutter, we show demonstration pairs from BBH, NLP-LR, and MMLU in Appendix J.

reproducibility. Overall, our evaluation spans a wide range of challenging tasks, model sizes from 1B to 8B parameters, varied numbers of demonstration pairs per task ($k = 2$ to $k = 16$), and benchmarks with and without task instructions (e.g., BBH includes instructions, while the others do not).

5.4 Comparing Context Tuning to Baselines

Table 1 reports the performance and training time per task for our baselines and methods across the four benchmarks. To fairly compare Prompt Tuning and Prefix Tuning with *Context Tuning*, “Prompt Tuning ($m = \#$ demo)” and “Prefix Tuning ($m = \#$ demo)” are configured to match the number of trainable parameters in *CT-Prompt* and *CT-KV*, respectively, by setting m to the number of demonstration pair tokens. We report each method’s number of trainable parameters in Appendix E.

Context Tuning outperforms Prompt Tuning, Prefix Tuning, and LoRA variants. *CT-Prompt* outperforms Prompt Tuning ($m = 32$), and *CT-KV* outperforms Prefix Tuning ($m = 32$), both by a wide margin across all benchmarks. Moreover, increasing m to match the number of demonstration tokens does not yield consistent improvements in Prompt Tuning or Prefix Tuning. Despite tuning the same number of parameters, these variants still underperform compared to *CT-Prompt* and *CT-KV*. In addition, *CT-KV*

outperforms LoRA, rank-stabilized LoRA (Kalajdzievski, 2023), and DoRA (Liu et al., 2024c). This highlights the effectiveness of leveraging the model’s ICL capabilities by initializing the prompt or prefix with demonstration tokens.

CT-KV is more efficient than CT-Prompt. *CT-KV* exhibits significantly lower training time per task compared to *CT-Prompt*. This observation aligns with the time complexity discussion in Appendix A: *CT-Prompt* incurs quadratic scaling in training time with the number of demonstration pairs, while *CT-KV* scales linearly. In addition to being faster, *CT-KV* also outperforms *CT-Prompt* in accuracy by conditioning each transformer layer’s activations with layer-specific key and value vectors, rather than relying solely on input-level soft prompts.

CT-KV offers an efficient alternative to TTT, and the two are complementary. *CT-KV* achieves performance comparable to TTT across NLP-LR, MMLU, and BBH, and solves the same number of ARC tasks. However, it requires at most half the training time per task compared to TTT on all benchmarks. This demonstrates that while the two methods converge to similar performance levels, *CT-KV* is more efficient due to its linear time complexity in the number of demonstration pairs, in contrast to TTT’s quadratic time complexity. Furthermore, *CT-KV* can be applied as a refinement step after TTT updates the model

Table 1. Few-shot learning performance on NLP-LR, MMLU, BBH, and ARC benchmarks. Each cell contains the accuracy (%) or training time per task (seconds), delimited by /. We show the means and standard deviations of accuracies over 5 seeds with different sets of demonstration pairs per task (except ARC because it has fixed demonstration pairs). The best accuracy is **bolded** and second best is underlined for each benchmark.

Method	NLP-LR		MMLU		BBH		ARC	
	Acc. (%)	T (s)	Acc. (%)	T (s)	Acc. (%)	T (s)	Acc. (%)	T (s)
Baselines								
Zero-Shot	34.9 $\pm$ 0.62	0	35.8 $\pm$ 0.71	0	40.9 $\pm$ 0.43	0	1.0	0
ICL	35.6 $\pm$ 0.65	0	41.2 $\pm$ 0.57	0	50.4 $\pm$ 0.78	0	13.3	0
LoRA	42.8 $\pm$ 0.88	156	40.1 $\pm$ 0.93	16	51.7 $\pm$ 0.66	9	13.5	14
Rank-Stabilized LoRA	41.7 $\pm$ 0.84	143	38.8 $\pm$ 0.87	17	46.7 $\pm$ 1.24	8	12.8	15
DoRA	42.9 $\pm$ 0.87	161	40.3 $\pm$ 0.90	16	52.6 $\pm$ 0.86	9	13.0	15
Prompt Tuning (m = 32)	41.4 $\pm$ 1.02	147	39.2 $\pm$ 1.04	15	50.8 $\pm$ 1.59	7	12.0	13
Prompt Tuning (m = # demo)	38.8 $\pm$ 1.23	231	37.3 $\pm$ 1.23	29	47.5 $\pm$ 1.84	16	14.5	49
Prefix Tuning (m = 32)	42.0 $\pm$ 0.85	123	39.9 $\pm$ 0.94	5	52.7 $\pm$ 1.12	7	9.3	14
Prefix Tuning (m = # demo)	41.1 $\pm$ 0.89	144	38.8 $\pm$ 0.81	8	52.8 $\pm$ 1.15	9	20.5	24
TTT	44.1 $\pm$ 0.65	342	43.6 $\pm$ 0.55	30	57.8 $\pm$ 1.13	14	<u>23.8</u>	56
Our Methods								
CT-Prompt	43.2 $\pm$ 0.61	228	43.6 $\pm$ 0.67	33	56.3 $\pm$ 0.98	14	22.5	52
CT-KV	<u>44.2 $\pm$ 0.55</u>	145	<u>43.7 $\pm$ 0.54</u>	9	<u>57.9 $\pm$ 0.78</u>	7	<u>23.8</u>	26
Combined Methods								
TTT+CT-KV	47.6 $\pm$ 0.53	372	44.1 $\pm$ 0.38	34	58.2 $\pm$ 0.73	17	25.8	63

weights, leading to higher performance on all benchmarks with minimal additional training time. This suggests that context and model-based adaptation methods within the *In-Context Optimization* framework are complementary and can be effectively combined for few-shot learning.

Initialization from demonstration pairs lowers standard deviation in performance. Initializing the trainable prompt or prefix from demonstration pairs, rather than from random tokens, reduces variance across random seeds in both *CT-Prompt* and *CT-KV*. This leads to more stable performance compared to Prompt Tuning and Prefix Tuning.

CT-KV outperforms MetaICL on NLP-LR. *CT-KV* achieves 44.2% accuracy on NLP-LR, surpassing the reported 43.3% accuracy of MetaICL on NLP-LR (Min et al., 2022a) when controlled for the same training and evaluation samples. This demonstrates that inference-time, single-task optimization with *CT-KV* can rival the performance of approaches that fine-tune model weights across many tasks.

5.5 Robustness to Demonstration Count and Quality

We now ablate the number of demonstration pairs (k) to assess *Context Tuning*’s robustness to it. We compare *CT-KV* to ICL and Prefix Tuning on NLP-LR and MMLU, varying $k = 4, 8, 12, 16$ for NLP-LR and $k = 16, 32, 48, 64$ for MMLU. Figure 4 (a) shows that across both benchmarks, *CT-KV* consistently outperforms ICL and Prefix Tuning

across k values and attains strong performance gains as k increases. Notably, despite ICL’s limited scalability with k on NLP-LR, *CT-KV* achieves greater relative performance gains with increasing k compared to both ICL and Prefix Tuning. We further evaluate *CT-KV* on BBH in the many-shot regime (Agarwal et al., 2024) at $k = 25, 50, 100$ in Appendix G, where its accuracy gap over ICL grows as k scales.

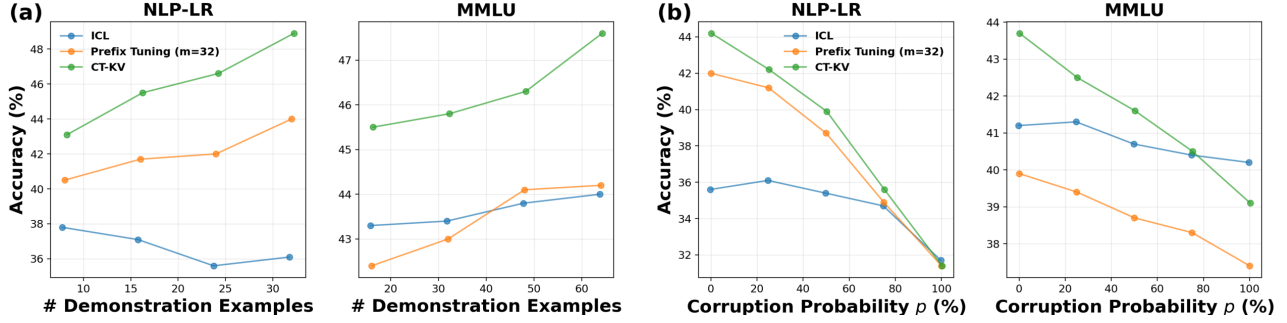
Next, we investigate *CT-KV*’s robustness to low-quality demonstration examples by injecting label noise. For NLP-LR and MMLU, we corrupt demonstration examples by replacing the multiple-choice option with an incorrect option with a corruption probability p . Figure 4 (b) shows that *CT-KV* achieves the best performance across both benchmarks for corruption rates up to 75%, demonstrating strong robustness to noisy demonstration examples.

5.6 Scaling Up the Pretrained Models

To broaden our model selections, we additionally evaluate *CT-KV*, ICL, and Prefix Tuning on BIG-Bench Hard (BBH) with large, recently released pretrained models: Mistral-NeMo-12B-Instruct (Mistral-AI-team, 2024), DeepSeek-R1-Distill-Qwen-14B and its 32B variant (DeepSeek-AI, 2025), and Qwen3-14B and its 32B variant (Qwen-Team, 2025). Table 2 shows that *CT-KV* significantly outperforms both ICL and Prefix Tuning, demonstrating its effectiveness across modern LLMs of

Table 2. Performance across large-scale pretrained models.

Method	Mistral-12B	DeepSeek-14B	DeepSeek-32B	Qwen3-14B	Qwen3-32B
ICL	51.3	55.2	63.5	60.8	63.1
Prefix Tuning (m = 32)	37.0	49.8	53.0	49.3	56.4
<i>CT-KV</i>	57.4	58.4	66.7	64.2	69.1

Figure 4. Performance of ICL, Prefix Tuning, and *CT-KV* under variations of (a) numbers of demonstration pairs and (b) probabilities of corrupted demonstration labels on NLP-LR and MMLU.

varying sizes and architectures. Notably, *CT-KV*'s gain over ICL reaches up to +6.1% (Mistral-12B), with consistent improvements across all five tested models.

5.7 Ablating Our Design Choices

We perform ablations on our design choices for *CT-KV*, namely Leave-One-Out Masking and Token Dropout. Table 3 shows that across all benchmarks, *CT-KV* without Token Dropout performs marginally worse than *CT-KV* with both components. This suggests that when tuning more parameters than traditional Prefix Tuning, applying dropout along the token dimension of Θ serves as an effective regularization technique for improving generalization. For NLP-LR, BBH, and MMLU, *CT-KV* performs significantly worse when Leave-One-Out Masking is not applied. This indicates that during training, it is crucial to mask out the portion of θ_{context} corresponding to the demonstration pair being solved, as it prevents the model from bypassing the task by retrieving the target output directly from the prefix initialization. However, on ARC, the model performs better without Leave-One-Out Masking. We hypothesize this is because ARC evaluation tasks typically include very few demonstration pairs (fewer than 4), so masking out even one pair during training can meaningfully reduce the effectiveness of the prompt or prefix in ICL. We also observe that when neither Leave-One-Out Masking nor Token Dropout is applied, *CT-KV* performs worse than ICL on MMLU and only marginally better on BBH, highlighting that these two design choices are essential to its overall performance. These design choices also act as regularizers that keep *CT-KV* stable under extended training, whereas Prefix Tuning overfits (Appendix H).

Table 3. Ablation study on the effects of Leave-One-Out Masking and Token Dropout in *CT-KV*.

Method	NLP-LR	MMLU	BBH	ARC
Neither	41.0 $\pm$ 0.75	40.2 $\pm$ 0.73	51.4 $\pm$ 0.76	21.0
No LOO Masking	42.6 $\pm$ 0.45	41.5 $\pm$ 0.65	54.4 $\pm$ 0.88	23.8
No Token Dropout	43.9 $\pm$ 0.62	42.7 $\pm$ 0.62	55.3 $\pm$ 0.72	21.0
Both	44.2 $\pm$ 0.55	43.7 $\pm$ 0.54	57.9 $\pm$ 0.78	22.5

5.8 Qualitative Results

We compare our *CT-KV* to ICL on the 400 ARC evaluation tasks. Figure 5 shows one failure case for each method, where the other successfully solves the task. The task at the top illustrates that *CT-KV* can effectively adapt to the demonstration pairs to solve a geometric puzzle involving cropping the upper-left portion of objects in the query. At the bottom, we show a case where *CT-KV* makes an incorrect prediction. Since *CT-KV* performs optimization on the 3 demonstration pairs and two of them, illustrated at the bottom of Figure 5, have answer grids that are 3-row by 4-column, we hypothesize that *CT-KV* became incorrectly biased toward predicting a grid of the same shape during optimization.

5.9 Why Does *CT-KV* Outperform ICL?

Two-Stage Interpretation of ICL. Table 1 shows that *CT-KV* significantly improves accuracy over ICL. To understand ICL's limitations, we frame it as a two-stage process: first, the model encodes task-relevant information from the demonstration pairs into an intermediate key-value (KV) cache via a forward pass, denoted as Θ_{CT} and used by *CT-KV* to initialize θ_{context} ; second, the model attends to this

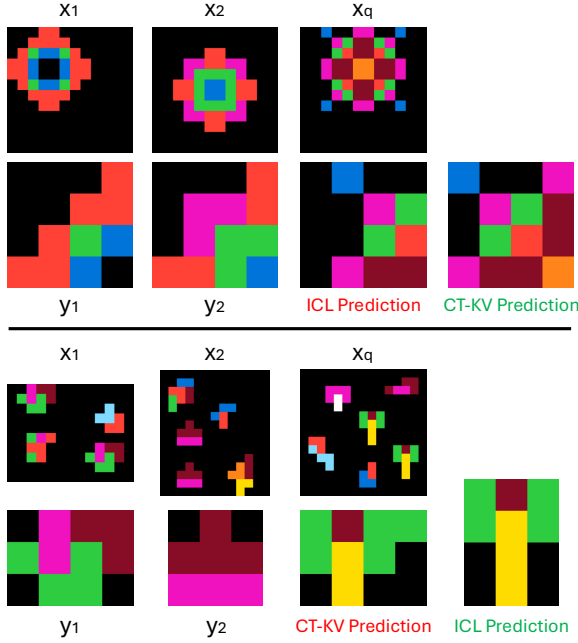


Figure 5. Top is an ARC task that *CT-KV* successfully solves, but ICL does not. Conversely, the task on the bottom is solved by ICL but not by *CT-KV*.

cache when generating an output for a new query input x_q .

Demonstration Pair Retrieval Experiment. Since the second stage does not revisit the original demonstration tokens, the KV cache must contain all necessary information to solve input-output pairs from the task, including the demonstration pairs themselves. To assess how well this information is encoded, we conduct a simple diagnostic: we concatenate all k demonstration pairs into the model’s context and then prompt it with the input from one of those same pairs. In this setup, the correct answer is already present in the context, so the model can either apply the task structure it has extracted from the other demonstration pairs or retrieve the correct output directly from the context.

Table 4 shows that even in this favorable setting, performance surprisingly remains far from perfect, suggesting that the KV cache often fails to encode the task adequately. *CT-KV* can be viewed as directly optimizing this KV cache Θ_{CT} by applying gradient updates on the demonstration pairs to refine the task representation. To prevent overfitting through memorization, we additionally use a Leave-One-Out Masking technique: when optimizing for a given demonstration pair, we exclude it from the context the model can attend to, forcing the model to generalize from the remaining pairs.

This perspective highlights a key weakness of ICL: relying on a single forward pass to encode complex task behavior often results in an incomplete or lossy task representation. In contrast, *CT-KV* uses gradient-based optimization to

Table 4. ICL accuracy on demonstration pairs with the same experiment setup as Section 5.3, but evaluating on demonstration pairs instead of query pairs.

NLP-LR	MMLU	BBH	ARC
81.9 ± 0.32	84.1 ± 0.45	89.3 ± 0.43	22.6

iteratively refine the cache by explicitly optimizing it to solve each demonstration pair, leading to a more effective and robust task encoding.

6 Conclusion

We introduce *Context Tuning*, a simple and effective method for improving few-shot learning in language models by directly optimizing a prompt or prefix initialized from demonstration tokens. Our method combines the strengths of ICL, which leverages pretrained knowledge by conditioning on task examples at inference time, and prompt-based adaptation, which efficiently adapts to new tasks by tuning a small number of parameters. We develop two versions of this approach: *CT-Prompt*, which tunes input-level soft prompts, and *CT-KV*, which tunes layer-specific key and value prefixes derived from the model’s activations on demonstration pairs. Across a broad set of benchmarks, both methods outperform ICL, traditional prompt-based tuning, and LoRA variants, with *CT-KV* offering a more favorable trade-off between performance and efficiency. Through ablation studies, we show that *CT-KV*’s performance depends critically on two design choices: Leave-One-Out Masking and Token Dropout. Additionally, we show that *CT-KV* is robust to varying demonstration count and quality, and scales to large pretrained models.

More broadly, we frame our method within the *In-Context Optimization* framework, which encompasses approaches that leverage in-context demonstration pairs to adapt either the model weights or its context at inference time. Our findings highlight that optimizing the lightweight context, rather than the model, is a powerful and scalable direction for few-shot learning. This approach achieves competitive performance with TTT in significantly less training time. Moreover, we show that *CT-KV* can be applied after TTT to improve performance further, suggesting that context and model adaptation can be effectively combined.

Limitations and Future Work. Section 5.8 identifies a potential limitation of *CT-KV*, where it may be prone to overfitting on certain tasks. Future directions to improve *CT-KV* include exploring stronger regularization techniques beyond Token Dropout, or applying KV cache compression techniques (Devoto et al., 2024; Ge et al., 2024; Liu et al., 2024a) to compress *CT-KV*’s initialization Θ before training, further improving overall efficiency.

Acknowledgments

This work was supported in part by Visko AI, a Google TPU Award, the NYU-KAIST Award A25-0081-002, and the Institute of Information & Communications Technology Planning Evaluation (IITP) under grant RS-2024-00469482, funded by the Ministry of Science and ICT (MSIT) of the Republic of Korea in connection with the Global AI Frontier Lab International Collaborative Research. The compute is supported by the NYU High Performance Computing resources, services, and staff expertise.

Impact Statement

Context Tuning enables language models to make more accurate predictions on novel tasks using only a few task-specific examples. Adapting to biased or harmful demonstrations risks propagating those properties into outputs, but this risk is no greater than that of other few-shot adaptation methods.

References

- Agarwal, R., Singh, A., Zhang, L. M., Bohnet, B., Rosias, L., Chan, S., Zhang, B., Anand, A., Abbas, Z., Nova, A., Co-Reyes, J. D., Chu, E., Behbahani, F., Faust, A., and Larochelle, H. Many-shot in-context learning. In *NeurIPS*, 2024.
- Akyürek, E., Damani, M., Zweiger, A., Qiu, L., Guo, H., Pari, J., Kim, Y., and Andreas, J. The surprising effectiveness of test-time training for few-shot learning. In *ICML*, 2025.
- Bonnet, C. and Macfarlane, M. V. Searching latent program spaces. *arXiv preprint arXiv:2411.08706*, 2024.
- Brown, T. B., Mann, B., Ryder, N., Subbiah, M., Kaplan, J., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., Agarwal, S., Herbert-Voss, A., Krueger, G., Henighan, T., Child, R., Ramesh, A., Ziegler, D. M., Wu, J., Winter, C., Hesse, C., Chen, M., Sigler, E., Litwin, M., Gray, S., Chess, B., Clark, J., Berner, C., McCandlish, S., Radford, A., Sutskever, I., and Amodei, D. Language models are few-shot learners. In *NeurIPS*, 2020.
- Chen, Y., Zhong, R., Zha, S., Karypis, G., and He, H. Meta-learning via language model in-context tuning. In *ACL*, 2022.
- Chollet, F. Abstraction and reasoning corpus for artificial general intelligence (ARC-AGI), 2019a. URL <https://github.com/fchollet/ARC-AGI>.
- Chollet, F. On the measure of intelligence. *arXiv preprint arXiv:1911.01547*, 2019b.
- Chollet, F., Knoop, M., Kamradt, G., and Landers, B. ARC prize 2024: Technical report. *arXiv preprint arXiv:2412.04604*, 2024.
- Dai, D., Sun, Y., Dong, L., Hao, Y., Ma, S., Sui, Z., and Wei, F. Why can GPT learn in-context? language models secretly perform gradient descent as meta-optimizers. In *ACL (Findings)*, 2023.
- Dalal, K., Kocejka, D., Xu, J., Zhao, Y., Han, S., Cheung, K. C., Kautz, J., Choi, Y., Sun, Y., and Wang, X. One-minute video generation with test-time training. In *CVPR*, 2025.
- DeepSeek-AI. DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.
- Deutch, G., Magar, N., Natan, T., and Dar, G. In-context learning and gradient descent revisited. In *NAACL*, 2024.
- Devoto, A., Zhao, Y., Scardapane, S., and Minervini, P. A simple and effective L_2 norm-based strategy for KV cache compression. In *EMNLP*, 2024.
- Dhariwal, P. and Nichol, A. Q. Diffusion models beat GANs on image synthesis. In *NeurIPS*, 2021.
- Franzen, D., Disselhoff, J., and Hartmann, D. Product of experts with LLMs: Boosting performance on ARC is a matter of perspective. In *ICML*, 2025.
- Gandelsman, Y., Sun, Y., Chen, X., and Efros, A. A. Test-time training with masked autoencoders. In *NeurIPS*, 2022.
- Garg, S., Tsipras, D., Liang, P., and Valiant, G. What can transformers learn in-context? a case study of simple function classes. In *NeurIPS*, 2022.
- Ge, S., Zhang, Y., Liu, L., Zhang, M., Han, J., and Gao, J. Model tells you what to discard: Adaptive KV cache compression for LLMs. In *ICLR*, 2024.
- Grattafiori, A., Dubey, A., Jauhri, A., Pandey, A., Kadian, A., Al-Dahle, A., Letman, A., Mathur, A., Schelten, A., Vaughan, A., Yang, A., Fan, A., Goyal, A., and et al. The Llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.
- Hardt, M. and Sun, Y. Test-time training on nearest neighbors for large language models. In *ICLR*, 2024.
- Hendrycks, D., Burns, C., Basart, S., Zou, A., Mazeika, M., Song, D., and Steinhardt, J. Measuring massive multitask language understanding. In *ICLR*, 2021.
- Ho, J. and Salimans, T. Classifier-free diffusion guidance. *arXiv preprint arXiv:2207.12598*, 2022.

- Ho, J., Jain, A., and Abbeel, P. Denoising diffusion probabilistic models. In *NeurIPS*, 2020.
- Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., and Chen, W. LoRA: Low-rank adaptation of large language models. In *ICLR*, 2022.
- Huang, Y., Qian, K., and Yu, Z. Learning a better initialization for soft prompts via meta-learning. In *IJCNLP-AACL*, 2023.
- Jang, J., Jang, S., Kweon, W., Jeon, M., and Yu, H. Rectifying demonstration shortcut in in-context learning. In *NAACL*, 2024.
- Jiang, A. Q., Sablayrolles, A., Mensch, A., Bamford, C., Chaplot, D. S., de las Casas, D., Bressand, F., Lengyel, G., Lample, G., Saulnier, L., Lavaud, L. R., Lachaux, M.-A., Stock, P., Scao, T. L., Lavril, T., Wang, T., Lacroix, T., and Sayed, W. E. Mistral 7B. *arXiv preprint arXiv:2310.06825*, 2023.
- Kalajdziewski, D. A rank stabilization scaling factor for fine-tuning with LoRA. *arXiv preprint arXiv:2312.03732*, 2023.
- Khashabi, D., Min, S., Khot, T., Sabharwal, A., Tafjord, O., Clark, P., and Hajishirzi, H. UNIFIEDQA: Crossing format boundaries with a single QA system. In *EMNLP (Findings)*, 2020.
- Kirkpatrick, J., Pascanu, R., Rabinowitz, N., Veness, J., Desjardins, G., Rusu, A. A., Milan, K., Quan, J., Ramalho, T., Grabska-Barwinska, A., Hassabis, D., Clopath, C., Kumaran, D., and Hadsell, R. Overcoming catastrophic forgetting in neural networks. *Proceedings of the National Academy of Sciences*, 2017.
- Lester, B., Al-Rfou, R., and Constant, N. The power of scale for parameter-efficient prompt tuning. In *EMNLP*, 2021.
- Li, X. and Qiu, X. Finding support examples for in-context learning. In *EMNLP (Findings)*, 2023.
- Li, X. L. and Liang, P. Prefix-tuning: Optimizing continuous prompts for generation. In *ACL*, 2021.
- Liu, A., Liu, J., Pan, Z., He, Y., Haffari, G., and Zhuang, B. MiniCache: KV cache compression in depth dimension for large language models. In *NeurIPS*, 2024a.
- Liu, J., Shen, D., Zhang, Y., Dolan, B., Carin, L., and Chen, W. What makes good in-context examples for GPT-3? In *DeeLIO Workshop at ACL*, 2022.
- Liu, S., Ye, H., Xing, L., and Zou, J. In-context vectors: making in context learning more effective and controllable through latent space steering. In *ICML*, 2024b.
- Liu, S.-Y., Wang, C.-Y., Yin, H., Molchanov, P., Wang, Y.-C. F., Cheng, K.-T., and Chen, M.-H. DoRA: Weight-decomposed low-rank adaptation. In *ICML*, 2024c.
- Liu, X., Ji, K., Fu, Y., Du, Z., Yang, Z., and Tang, J. P-Tuning v2: Prompt tuning can be comparable to fine-tuning universally across scales and tasks. *arXiv preprint arXiv:2110.07602*, 2021.
- Liu, X., Zheng, Y., Du, Z., Ding, M., Qian, Y., Yang, Z., and Tang, J. GPT understands, too. *AI Open*, 2024d.
- Lu, J., Teehan, R., and Ren, M. Procreate, don't reproduce! propulsive energy diffusion for creative generation. In *ECCV*, 2024.
- Min, S., Lewis, M., Zettlemoyer, L., and Hajishirzi, H. MetalCL: Learning to learn in context. In *NAACL*, 2022a.
- Min, S., Lyu, X., Holtzman, A., Artetxe, M., Lewis, M., Hajishirzi, H., and Zettlemoyer, L. Rethinking the role of demonstrations: What makes in-context learning work? In *EMNLP*, 2022b.
- Mistral-AI-team. Mistral NeMo, 2024. URL <https://mistral.ai/news/mistral-nemo>.
- Muhtar, D., Shen, Y., Yang, Y., Liu, X., Lu, Y., Liu, J., Zhan, Y., Sun, H., Deng, W., Sun, F., Zhang, X., Gao, J., Chen, W., and Zhang, Q. StreamAdapter: Efficient test time adaptation from contextual streams. *arXiv preprint arXiv:2411.09289*, 2024.
- Nichol, A. Q., Dhariwal, P., Ramesh, A., Shyam, P., Mishkin, P., McGrew, B., Sutskever, I., and Chen, M. GLIDE: towards photorealistic image generation and editing with text-guided diffusion models. In *ICML*, 2022.
- Pan, K., Li, J., Song, H., Lin, J., Liu, X., and Tang, S. Self-supervised meta-prompt learning with meta-gradient regularization for few-shot generalization. In *EMNLP (Findings)*, 2023.
- Qin, C., Joty, S., Li, Q., and Zhao, R. Learning to initialize: Can meta learning improve cross-task generalization in prompt tuning? In *ACL*, 2023.
- Qwen-Team. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.
- Radford, A., Wu, J., Child, R., Luan, D., Amodei, D., and Sutskever, I. Language models are unsupervised multitask learners. *OpenAI*, 2019.
- Rombach, R., Blattmann, A., Lorenz, D., Esser, P., and Ommer, B. High-resolution image synthesis with latent diffusion models. In *CVPR*, 2022.

- Shin, T., Razeghi, Y., IV, R. L. L., Wallace, E., and Singh, S. AutoPrompt: Eliciting knowledge from language models with automatically generated prompts. In *EMNLP*, 2020.
- Singhal, K., Azizi, S., Tu, T., Mahdavi, S. S., Wei, J., Chung, H. W., Scales, N., Tanwani, A., Cole-Lewis, H., Pfohl, S., Payne, P., Seneviratne, M., Gamble, P., Kelly, C., Scharli, N., Chowdhery, A., Mansfield, P., y Arcas, B. A., Webster, D., Corrado, G. S., Matias, Y., Chou, K., Gottweis, J., Tomasev, N., Liu, Y., Rajkomar, A., Barral, J., Semturs, C., Karthikesalingam, A., and Natarajan, V. Large language models encode clinical knowledge. *Nature*, 2023.
- Srivastava, A., Rastogi, A., Rao, A., Md-Shoeb, A.-A., Abid, A., Fisch, A., Brown, A., Santoro, A., Gupta, A., and et al. Beyond the imitation game: Quantifying and extrapolating the capabilities of language models. *Transactions on Machine Learning Research*, 2023.
- Sun, Y., Wang, X., Liu, Z., Miller, J., Efros, A. A., and Hardt, M. Test-time training with self-supervision for generalization under distribution shifts. In *ICML*, 2020.
- Suzgun, M., Scales, N., Scharli, N., Gehrmann, S., Tay, Y., Chung, H. W., Chowdhery, A., Le, Q. V., Chi, E. H., Zhou, D., and Wei, J. Challenging BIG-Bench tasks and whether chain-of-thought can solve them. In *ACL (Findings)*, 2023.
- Wallace, B., Gokul, A., Ermon, S., and Naik, N. End-to-end diffusion latent optimization improves classifier guidance. In *ICCV*, 2023.
- Wang, X., Wei, J., Schuurmans, D., Le, Q. V., Chi, E. H., Narang, S., Chowdhery, A., and Zhou, D. Self-consistency improves chain of thought reasoning in language models. In *ICLR*, 2023.
- Wei, J., Wang, X., Schuurmans, D., Bosma, M., Ichter, B., Xia, F., Chi, E. H., Le, Q. V., and Zhou, D. Chain-of-thought prompting elicits reasoning in large language models. In *NeurIPS*, 2022.
- Ye, Q., Lin, B. Y., and Ren, X. CrossFit: A few-shot learning challenge for cross-task generalization in NLP. In *EMNLP*, 2021.
- Zhao, J. In-context exemplars as clues to retrieving from large associative memory. In *ICML Neural Conversational AI*, 2023.

Appendix

A Time Complexity

At each training iteration, an LLM’s forward and backward passes are dominated by its self-attention operations. Consider a single attention head of dimension d . Let L_Q denote the number of query tokens and L_K the number of key (and value) tokens. We form the query matrix $\mathbf{Q} \in \mathbb{R}^{L_Q \times d}$ and the key and value vectors $\mathbf{K}, \mathbf{V} \in \mathbb{R}^{L_K \times d}$, then compute

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d}}\right)\mathbf{V},$$

whose dominant cost is the matrix multiplication $\mathbf{Q}\mathbf{K}^\top$, requiring $O(L_Q L_K d)$ operations per head. Because d is a constant for a given model, we omit it in our comparisons below.

Next, let n be the number of tokens in the task’s query and p the number of additional trainable prompt or prefix tokens per layer. We analyze how the training time of each method in the *In-Context Optimization* framework scales with n and p .

Test-Time Training. At each layer of each training iteration, TTT prepends p trainable tokens to the n query tokens and computes their keys and values, giving $L_Q = n + p$ and $L_K = n + p$ with a per-head cost of

$$O((n + p)^2).$$

CT-Prompt. *CT-Prompt* prepends p trainable soft token embeddings to the query and computes their keys and values, also giving $L_Q = n + p$ and $L_K = n + p$ with a per-head cost of

$$O((n + p)^2).$$

CT-KV. Unlike TTT and *CT-Prompt*, *CT-KV* prepends p trainable tokens as past keys and values, so these tokens do not generate queries. This yields $L_Q = n$ and $L_K = n + p$ with a per-head cost of only

$$O(n(n + p)).$$

Time Complexity for k Demonstrations. Suppose we have k demonstration pairs, each of length ℓ (assuming equal length). In TTT, $n = \ell$ is the length of a demonstration pair and $p = (k - 1)\ell$ is the summed length of other demonstration pairs. For *CT-Prompt* and *CT-KV*, n and p have the same values as TTT because Leave-One-Out Masking masks out one of the in-context demonstration pairs. Table 5 summarizes the per-head costs in k and ℓ , showing that both *CT-Prompt* and TTT incur quadratic cost in k , while *CT-KV*’s cost grows only linearly in k . This k -fold reduction in self-attention complexity explains *CT-KV*’s faster empirical training speed in Table 1.

Table 5. Per-head self-attention time complexity for methods with k demonstration pairs of length ℓ .

Method	L_Q	L_K	Per-Head Cost
TTT	$k\ell$	$k\ell$	$O((k\ell)^2)$
CT-Prompt	$k\ell$	$k\ell$	$O((k\ell)^2)$
CT-KV	ℓ	$k\ell$	$O(k\ell^2)$

B Prompt Tuning and Prefix Tuning with Other Initialization Schemes

In Table 1, we reported Prompt Tuning and Prefix Tuning results using only random-token initialization for their trainable prompts and prefixes. Here, we also follow [Lester et al. \(2021\)](#) in initializing prompts from a uniform distribution, and [Li & Liang \(2021\)](#) in initializing prefixes either from a uniform distribution or from a seed prefix passed through a two-layer MLP (hidden size 512). Table 6 shows that both Prompt Tuning and Prefix Tuning perform best with random-token initialization, confirming the findings of those works. Therefore, even when compared against these alternative initialization schemes, *CT-Prompt* and *CT-KV* continue to deliver superior performance.

Table 6. Ablation of initialization schemes for Prompt Tuning and Prefix Tuning. We show the means of accuracies over 5 seeds with different sets of demonstration pairs per task (except for ARC because it has fixed demonstration pairs).

Method	NLP-LR	MMLU	BBH	ARC
Prompt Tuning ($m = 32$, uniform)	39.4	34.3	34.4	5.0
Prompt Tuning ($m = 32$, token)	41.4	39.2	50.8	12.0
Prefix Tuning ($m = 32$, uniform)	38.2	26.8	10.22	3.3
Prefix Tuning ($m = 32$, MLP)	39.6	26.2	11.03	7.3
Prefix Tuning ($m = 32$, token)	42.0	39.9	52.7	9.3

C More Details on Experiment Setup

We detail below our hyperparameter settings for the results reported in Table 1 and Table 6. For TTT, we follow Akyürek et al. (2025): using a LoRA learning rate of $1e-4$, sampling a random permutation of the k demonstration pairs at each training step, and setting the LoRA rank to 128 for ARC and 64 for all other tasks. In our TTT+CT-KV experiments, we find that using a small number of CT-KV training iterations and lower learning rates further boosts performance on top of a TTT-adapted model.

For all other experiments, we search over learning rates $3e-4$, $1e-3$, $3e-3$ and Token Dropout rates 0, 0.05, 0.1. We search training iterations 150, 200, 250, 300 for NLP-LR and ARC experiments, 15, 20, 25, 30 for MMLU experiments, and 12, 16, 20, 24 for BBH experiments. Table 7 shows our hyperparameter choices. For fair comparison, hyperparameter sweeps are performed for all methods. For CT-Prompt, CT-KV, and TTT+CT-KV, we use Token Dropout rates of 0.05 for NLP-LR and 0.1 for MMLU, BBH, and ARC.

Experiments for NLP-LR are performed on a single RTX8000, while all other experiments are conducted on a single A100. All experiments use the Adam optimizer, a cosine learning rate scheduler with no warm-up, bfloat16 precision, and up to 32GB of CPU RAM.

To fairly compare efficiency, we train each method with the largest batch size possible for the GPU used in its experiment. Since TTT, Prompt Tuning ($m = \# \text{ demo}$), and CT-Prompt use more memory than other methods due to computing larger QK^T matrices (as shown in our derivation in Section A), we limit their batch sizes to 4 for NLP-LR, MMLU, and ARC, and 5 for BBH. MMLU and BBH models use gradient checkpointing. For all other methods, we use batch size 16 for NLP-LR, 8 for MMLU with gradient accumulation of 2, 2 for BBH with gradient accumulation of 5, and full batch for ARC (depending on each task’s number of demonstration pairs). All models, unless noted, do not require gradient checkpointing.

Table 7. Learning rates (LR) and number of training iterations (#iters) used for each method and benchmark.

Method	NLP-LR		MMLU		BBH		ARC	
	LR	#iters	LR	#iters	LR	#iters	LR	#iters
Prompt Tuning ($m = 32$, uniform)	$3e-3$	200	$3e-3$	25	$1e-3$	20	$3e-3$	250
Prompt Tuning ($m = 32$, token)	$3e-3$	200	$1e-3$	25	$3e-3$	16	$3e-3$	200
Prompt Tuning ($m = \# \text{ demo}$, token)	$1e-3$	250	$1e-3$	20	$3e-4$	16	$3e-3$	200
Prefix Tuning ($m = 32$, uniform)	$3e-3$	250	$3e-3$	25	$1e-3$	20	$3e-3$	250
Prefix Tuning ($m = 32$, MLP)	$3e-3$	250	$1e-3$	25	$3e-3$	20	$1e-3$	200
Prefix Tuning ($m = 32$, token)	$3e-3$	250	$3e-3$	25	$3e-3$	16	$3e-3$	200
Prefix Tuning ($m = \# \text{ demo}$, token)	$1e-3$	250	$3e-3$	25	$3e-3$	16	$3e-3$	200
CT-Prompt	$1e-3$	250	$1e-3$	25	$3e-4$	12	$1e-3$	250
CT-KV	$1e-3$	200	$3e-3$	20	$1e-3$	16	$3e-3$	200
TTT	$1e-4$	250	$1e-4$	20	$1e-4$	8	$1e-4$	200
TTT+CT-KV	$1e-3$	25	$1e-4$	5	$1e-3$	8	$1e-3$	25

D Parameter-Efficient Variants of CT-KV

In this section, we explore two variants of *CT-KV* that reduce the number of trainable prefix parameters. We use the notations from Section 4.

CT-V. We partition the trainable prefix Θ_{CT} into its key and value components, Θ_K and Θ_V . Inspired by Kirkpatrick et al. (2017), we estimate the importance of each trainable parameter $\Theta_j \in \Theta_{CT}$ by computing its diagonal Fisher term over the k demonstrations in $\mathcal{D}$:

$$\hat{F}_j = \frac{1}{k} \sum_{i=1}^k (\nabla_{\Theta_j} \log p_{\phi}(y_i \mid \Theta_{CT}, x_i))^2.$$

$\hat{F}_j$ provides a relative estimate of how much a change in each parameter Θ_j affects the model’s ability to solve each demonstration pair, representing its importance during training. By averaging $\hat{F}_j$ across parameters in Θ_K and Θ_V , we obtain two scalar estimates, $\hat{F}_K$ and $\hat{F}_V$, indicating the relative importance of the trainable keys and values, respectively. Based on our findings in Table 8, we conclude that $\hat{F}_V \gg \hat{F}_K$ for most tasks, suggesting that values play a more significant role. By freezing $\Theta_K \subset \Theta_{CT}$ and training only $\Theta_V \subset \Theta_{CT}$, we arrive at *CT-V*, which reduces the number of trainable parameters in *CT-KV* by exactly half.

Table 8. Average Fisher information for the trainable key parameters $\Theta_K \subset \Theta_{CT}$ and value parameters $\Theta_V \subset \Theta_{CT}$ across 5 random selections of k demonstration pairs over each dataset.

Dataset	$\hat{F}_K$	$\hat{F}_V$
ARC	2.43×10^{-9}	1.02×10^{-7}
BBH	1.89×10^{-6}	3.99×10^{-4}
NLP-LR	1.44×10^{-8}	8.32×10^{-8}
MMLU	2.81×10^{-8}	1.42×10^{-6}

CT-Prefix. We freeze Θ_{CT} , average the parameters across tokens to obtain an average prefix $\bar{\Theta}_{CT}$, and then form a new trainable m -token prefix Θ_{prefix} by adding small Gaussian perturbations:

$$\Theta_{\text{prefix}} = \{\bar{\Theta}_{CT} + \epsilon_i\}_{i=1}^m,$$

where $\epsilon_i \in \mathcal{N}(0, 0.02)$. The model additionally conditions on Θ_{prefix} , analogous to Prefix Tuning. Since we only train Θ_{prefix} , this variant has the same number of trainable parameters as Prefix Tuning with m tokens.

We evaluate *CT-V* and *CT-Prefix* across all benchmarks and compare them to *CT-KV* in Table 9, showing that both parameter-efficient variants retain most of the performance gain of *CT-KV* and outperform Prefix Tuning. For *CT-V*, we use the same hyperparameters as *CT-KV* from Section C. For *CT-Prefix*, we find that higher learning rates, 1e-1 for NLP-LR and 5e-2 for other datasets, are needed for better performance.

Table 9. Accuracies (%) of *CT-KV*, its parameter-efficient variants, and Prefix Tuning across benchmarks, averaged over 5 seeds (except for ARC because it has fixed demonstration pairs).

Method	NLP-LR	MMLU	BBH	ARC
Prefix Tuning (m = 32)	42.0	39.9	52.7	9.3
<i>CT-Prefix</i>	44.0	42.6	55.9	22.8
<i>CT-V</i>	44.0	43.5	57.5	23.5
<i>CT-KV</i>	44.2	43.7	57.9	23.8

E Number of Trainable Parameters

Corresponding to the performance shown in Table 1, we report the average number of trainable parameters for each method across tasks in Table 10. Note that although *CT-KV*’s number of trainable parameters scales with the number of demonstration tokens, it still trains significantly fewer parameters on average per task than the number of LoRA parameters

used by TTT. Following Akyürek et al. (2025), we use task instructions for BBH and set the instruction prompt or prefix to be trainable as well. We omit Zero-Shot and ICL from this comparison because they do not involve any trainable parameters.

Table 10. Number of trainable parameters (in thousands) for each method across benchmarks, corresponding to entries in Table 1.

Method	NLP-LR	MMLU	BBH	ARC
Prompt Tuning (m = 32)	41	98	229	66
Prompt Tuning (m = # demo)	578	2160	3656	2743
Prefix Tuning (m = 32)	2949	1835	3668	524
Prefix Tuning (m = # demo)	41634	40327	58501	21944
TTT	47186	89915	157286	84935
<i>CT-Prompt</i>	578	2160	3656	2743
<i>CT-Prefix</i>	2949	1835	3668	524
<i>CT-V</i>	20817	20163	29250	10972
<i>CT-KV</i>	41634	40327	58501	21944
TTT+ <i>CT-KV</i>	88820	130242	215787	106878

F Memory Profiling

We compare the training-time memory usage of *CT-KV* and our baselines in Table 1 in the table below. Values indicate the peak GPU memory usage (MB) when training with batch size 1 for one epoch with no gradient accumulation or checkpointing. Results are averaged across all tasks of each benchmark. We omit Zero-Shot and ICL because they do not require training. Since TTT+*CT-KV* simply runs the two methods sequentially, its memory usage would be the maximum of TTT and *CT-KV*. All other experimental settings remain identical to those in Table 1.

Table 11. Memory profiling *CT-KV* and baselines.

Method	NLP-LR	MMLU	BBH	ARC
Prompt Tuning (m = 32)	3601	13051	31640	4486
Prompt Tuning (m = # demo)	7319	18868	37493	6451
Prefix Tuning (m = 32)	3588	13220	31549	4598
Prefix Tuning (m = # demo)	5648	15570	33292	5340
TTT	8883	18690	54752	7232
<i>CT-Prompt</i>	7318	18911	37571	6629
<i>CT-KV</i>	5648	15569	33292	5340

Together with the accuracy results in Table 1, our memory profiling shows that *CT-KV* uses less memory than TTT, outperforms Prefix Tuning (m = # demo) at the same memory usage, and incurs less than 20% additional memory overhead compared to Prefix Tuning (m = 32) on MMLU, BBH, and ARC, while achieving significantly higher accuracy.

G Many-Shot Performance on BBH

We extend the demonstration-count ablation in Section 5.5 to the many-shot regime on BBH using Llama3-8B-Instruct, restricted to the 8 tasks whose 100-shot prompts fit within the 8k context window. Hyperparameters follow Table 1 except for a sweep over training epochs.

ICL degrades from 48.7% at 25-shot to 46.5% at 100-shot, while *CT-KV* keeps improving as demonstrations are added, widening its gap over ICL from +6.6% to +16.1% by 100-shot.

H Stability Under Extended Training

We sweep the number of training epochs on BBH using the Table 1 setup to compare *CT-KV* against Prefix Tuning under prolonged optimization. The ICL baseline accuracy on BBH is 50.44%.

Table 12. Many-shot performance of ICL and *CT-KV* on BBH with Llama3-8B-Instruct.

Method	Epochs	25-shot	50-shot	100-shot
ICL	—	48.7	46.8	46.5
<i>CT-KV</i>	3	50.5	51.6	51.2
	10	52.6	56.1	57.5
	20	54.0	58.6	61.1
	40	54.9	59.8	62.6
	60	55.3	59.0	62.0

Table 13. BBH accuracy of *CT-KV* and Prefix Tuning as the number of training epochs increases.

Epochs	4	8	16	32	64
Prefix Tuning	42.15	49.35	51.00	52.73	50.49
<i>CT-KV</i>	53.13	55.46	57.91	57.93	57.88

CT-KV’s accuracy is essentially flat from epoch 16 to 64 (57.91 $\rightarrow$ 57.88). Prefix Tuning peaks at epoch 32 (52.73) and declines by epoch 64 (50.49), falling back to the ICL baseline despite tuning fewer parameters than *CT-KV*.

I Qualitative Samples vs. Training Iteration

In this section, we select sample tasks from question-answering datasets to illustrate how autoregressively generated answers gradually improve with *CT-KV* training. We present two ARC tasks in Figure 6. In the top task, the model’s prediction at iteration 0 (equivalent to ICL) shows a strong bias toward filling orange squares with yellow. As *CT-KV* training progresses, the model gradually learns to fill each orange square with the correct color. Similarly, in the bottom task, the model first learns that only grey grid cells can turn red, and then correctly completes the cross shapes.

Similarly, for BBH, in Figure 7’s top query, the model initially predicts “padre, panicking” and “schoolmate, suburbia” in reversed order at iteration 0. During *CT-KV* training, the model learns to use the second letter of each word for sorting and eventually answers the query correctly. Likewise, for the bottom query, *CT-KV* helps the model avoid omitting the word “scrumptious” from its outputs and sort the words “sidereal, siena” into the correct order based on their second letters.

J Demonstration Pairs for Figure 3

We present three demonstration pairs of datasets: BBH, NLP-LR, and MMLU in Figure 8, Figure 9, and Figure 10, respectively.

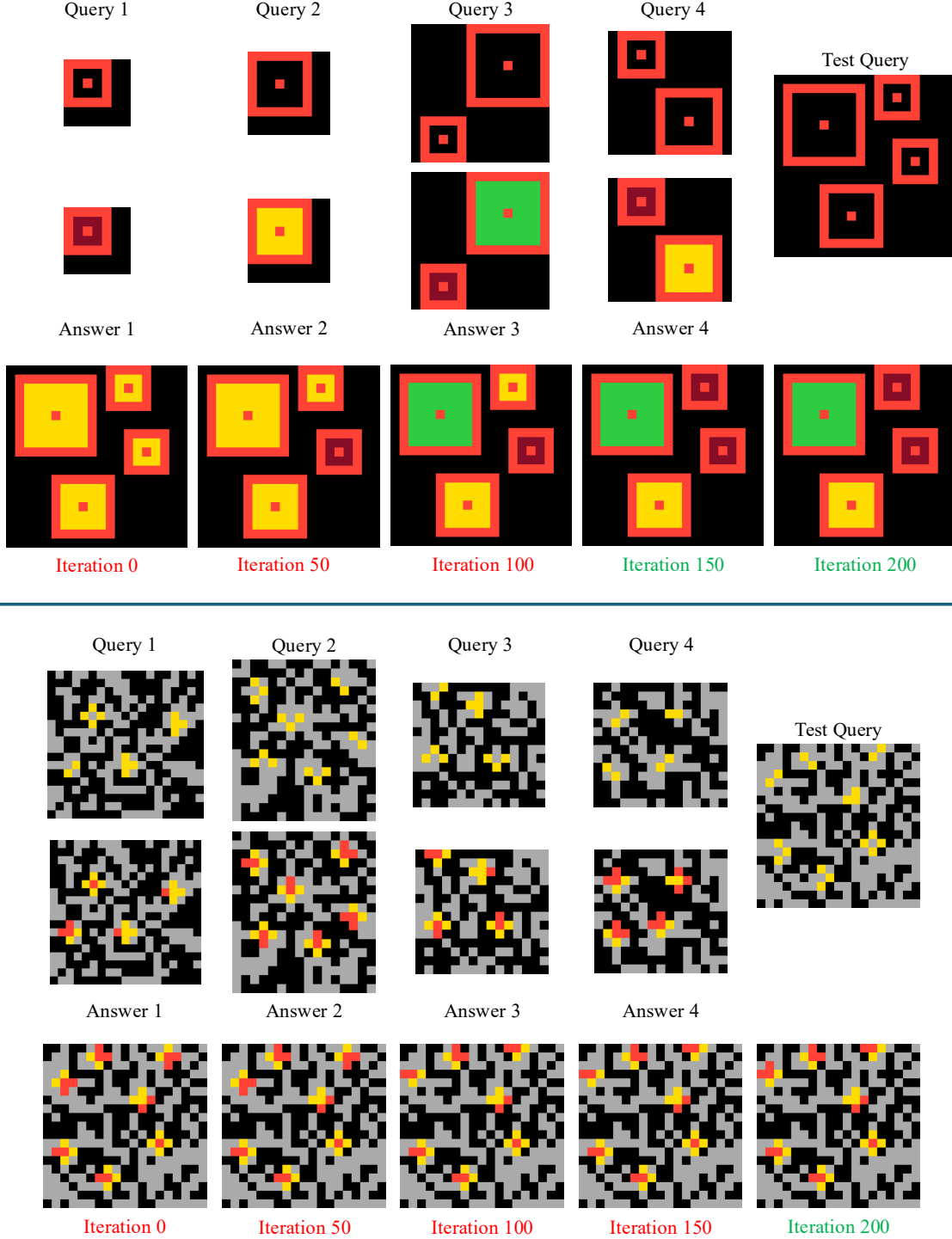


Figure 6. For each of the two ARC tasks at the top and bottom, we display 4 demonstration query-answer pairs, the test query, and LLM predictions at *CT-KV* training iterations 0, 50, 100, 150, 200. Note that iteration 0 is equivalent to ICL. We color-code the iterations of correct predictions in green and incorrect predictions in red.

Instruction: Sort a list of words. Answer with only a sequence of words.

Query: Sort the following words alphabetically: List: wv habitation panicking schoolmate aerodynamic giacomo nebulous offset vector roosevelt jimmy botanist suburbia padre

Iteration 0: aerodynamic botanist giacomo habitation jimmy nebulous offset panicking padre roosevelt suburbia schoolmate vector wv

Iteration 12: aerodynamic botanist giacomo habitation jimmy nebulous offset panicking padre roosevelt suburbia schoolmate vector wv

Iteration 16: aerodynamic botanist giacomo habitation jimmy nebulous offset panicking padre roosevelt schoolmate suburbia vector wv

Iteration 20: aerodynamic botanist giacomo habitation jimmy nebulous offset padre panicking roosevelt schoolmate suburbia vector wv

Instruction: Sort a list of words. Answer with only a sequence of words.

Query: Sort the following words alphabetically: List: scrumptious sidereal thermal yakima siena gorky saxon scottish figural hydroxyl seventeen neapolitan rampage nerve grapple fate plainfield stooge knives allotted

Iteration 0: allotted fate figural gorky grapple hydroxyl knives neapolitan nerve plainfield rampage saxon scottish seventeen siena sidereal stooge thermal yakima

Iteration 12: allotted fate figural gorky grapple hydroxyl knives neapolitan nerve plainfield rampage saxon scottish seventeen siena sidereal stooge thermal yakima

Iteration 16: allotted fate figural gorky grapple hydroxyl knives neapolitan nerve plainfield rampage saxon scottish scrumptious seventeen siena sidereal stooge thermal yakima

Iteration 20: allotted fate figural gorky grapple hydroxyl knives neapolitan nerve plainfield rampage saxon scottish scrumptious seventeen sidereal siena stooge thermal yakima

Figure 7. We display LLM predictions at *CT-KV* training iterations 0, 12, 16, 20 for two queries from the task “word sorting” in BBH. We omit showing the 16 demonstration pairs of each task for brevity. We color-code the iterations of correct predictions in green and incorrect predictions in red.

(1)	Instruction: A logical deduction task which requires deducing the order of a sequence of objects. Answer with only the corresponding letter (e.g. (A)). Query: The following paragraphs each describe a set of three objects arranged in a fixed order. The statements are logically consistent within each paragraph. In an antique car show, there are three vehicles: a motorcycle, a limousine, and a convertible. The motorcycle is newer than the limousine. The convertible is newer than the motorcycle. Options: (A) The motorcycle is the oldest (B) The limousine is the oldest (C) The convertible is the oldest Answer: (B)
(2)	Instruction: A logical deduction task which requires deducing the order of a sequence of objects. Answer with only the corresponding letter (e.g. (A)). Query: The following paragraphs each describe a set of three objects arranged in a fixed order. The statements are logically consistent within each paragraph. On a shelf, there are three books: a blue book, an orange book, and a red book. The blue book is the rightmost. The orange book is the leftmost. Options: (A) The blue book is the second from the left (B) The orange book is the second from the left (C) The red book is the second from the left Answer: (C)
(3)	Instruction: A logical deduction task which requires deducing the order of a sequence of objects. Answer with only the corresponding letter (e.g. (A)). Query: The following paragraphs each describe a set of three objects arranged in a fixed order. The statements are logically consistent within each paragraph. In an antique car show, there are three vehicles: a motorcycle, a minivan, and a tractor. The minivan is older than the tractor. The minivan is the second-newest. Options: (A) The motorcycle is the newest (B) The minivan is the newest (C) The tractor is the newest Answer: (C)

Figure 8. 3 demonstration pairs for the BBH task from Figure 3.

(1)	Query: Cellular respiration releases Options: <i>blood, waste, snot, feces</i> Answer: <i>waste</i>
(2)	Query: During what period of the Earth cycle would you see someone having a picnic outside? Options: <i>Day, Night, Extinction, Ice Age</i> Answer: <i>Day</i>
(3)	Query: Which uses gills to breathe? Options: <i>hermit crab, human, blue whale, bluebird</i> Answer: <i>hermit crab</i>

Figure 9. 3 demonstration pairs for the NLP-LR task from Figure 3.

(1)	Query: The inverse of $-i$ in the multiplicative group, $\{1, -1, i, -i\}$ is Options: <i>1, -1, i, -i</i> Answer: <i>i</i>
(2)	Query: Find the degree for the given field extension $\mathbb{Q}(\sqrt{2}, \sqrt{3}, \sqrt{18})$ over $\mathbb{Q}$. Options: <i>0, 4, 2, 6</i> Answer: <i>4</i>
(3)	Query: Find the order of the factor group $(\mathbb{Z}_{11} \times \mathbb{Z}_{15})/(\langle 1, 1 \rangle)$ Options: <i>1, 2, 5, 11</i> Answer: <i>1</i>

Figure 10. 3 demonstration pairs for the MMLU task from Figure 3.